

Session 14 — File Handling

- So far our programs worked with data stored in variables.
- But when the program stops, that data disappears.
- In real applications we need to store data permanently.

Examples

- Saving user information
- Reading configuration files
- Processing datasets
- Reading logs

For this purpose we use files.

Python allows us to read and write files very easily.

Types of files we will work with

- Text files
- CSV files
- JSON files

✦ Opening a File

In Python we use the `open()` function to work with files.

Syntax

```
file_object = open("filename", mode)
```

filename → name of the file

mode → how we want to use the file

Common modes

r → read file

w → write file (creates new file or overwrites)

a → append to file

```
# Writing to a text file

file = open("sample.txt", "w")

file.write("Welcome to Python File Handling\n")
file.write("This file was created by Python\n")
file.write("This file was created by Python\n")
file.write("This file was created by Python")

file.close()
```

✦ Reading a Text File

After data is stored in a file, we can read it.

Methods used

`read()` → reads entire file

`readline()` → reads one line

`readlines()` → reads all lines as a list

```
# Reading a text file
```

```

file = open("sample.txt", "r")

content = file.read()

print(content)

file.close()

```

```

Welcome to Python File Handling
This file was created by Python
This file was created by Python
This file was created by Python

```

✓ Using 'with' Statement (Best Practice)

Instead of manually closing files, Python provides the with statement.

It automatically closes the file after the block ends.

```

# Recommended way of handling files

with open("sample.txt", "r") as file:
    data = file.read()
    print(data)

```

```

Welcome to Python File Handling
This file was created by Python
This file was created by Python
This file was created by Python

```

Session 15 — Error and Exception Handling

Programs do not always run perfectly.

Users may enter invalid values.

Examples

Dividing by zero

Entering text instead of number

File not found

These situations cause errors.

Python allows us to handle these safely using exception handling.

✓ try and except

We place risky code inside try.

If an error occurs, except block runs instead of crashing the program.

```

num = int(input("Enter a number: "))
print("You entered:", num)

```

Enter a number: 12A

```

-----
ValueError                                Traceback (most recent call last)
/tmp/ipykernel_13023/684549172.py in <cell line: 0>()
----> 1 num = int(input("Enter a number: "))
      2 print("You entered:", num)

ValueError: invalid literal for int() with base 10: '12A'

```

Next steps: [Explain error](#)

```

try:
    num = int(input("Enter a number: "))
    print("You entered:", num)

```

```
except:
    num = input("Enter a number: ")
    print("You entered:", num)

finally:
    print("Invalid input")
```

```
Enter a number: 5
You entered: 5
Invalid input
```

```
try:
    nu = int(input("Enter a number: "))
    print("You entered:", nu)

except:
    nu = input("Enter a number: ")
    print("You entered:", nu)

finally:
    print("Invalid input")
```

```
Enter a number: 12
You entered: 12
Invalid input
```

▼ Specific Exceptions

We can catch specific errors.

Example

- ZeroDivisionError
- ValueError

```
try:
    a = 10
    b = int(input("Enter divisor: "))

    result = a / b

    print("Result:", result)

except ZeroDivisionError:
    print("Cannot divide by zero")
```

```
Enter divisor: 0
Cannot divide by zero
```

▼ else Block

The else block runs if no exception occurs.

```
try:
    num = int(input("Enter number: "))

except ValueError:
    print("Invalid number")

else:
    print("Valid input:", num)
```

```
Enter number: hg
Invalid number
```

▼ finally Block

finally always runs regardless of error.

It is used for cleanup tasks.

```
try:
    file = open("sampllle.txt")

except FileNotFoundError:
    print("File not found")

finally:
    print("Program finished")
```

```
File not found
Program finished
```

✓ Raising Custom Exceptions

Sometimes we want to create our own error conditions.

```
age = int(input("Enter age: "))

if age < 0:
    raise ValueError("Age cannot be negative")
```

```
Enter age: 5
```

```
try:
    a = int(input("Enter first number: "))
    b = int(input("Enter second number: "))

    result = a / b

    print("Result:", result)

except ZeroDivisionError:
    print("Division by zero is not allowed")

except ValueError:
    print("Invalid input. Enter numbers only")

finally:
    print("Division program completed")
```

```
Enter first number: 40
Enter second number: 0
Division by zero is not allowed
Division program completed
```

✓ Error and Exception Handling

These questions help students understand how Python handles errors and how programs can be made safer using exception handling.

1. Write a program that takes a number as input and prints the number.
Use try and except to handle the situation where the user enters text instead of a number.
2. Write a program that divides two numbers entered by the user.
Use try and except to handle the ZeroDivisionError if the user tries to divide by zero.
3. Write a program that asks the user to enter an integer.
If the user enters an invalid value, display the message "Invalid number entered".
4. Write a program that tries to open a file called data.txt.
If the file does not exist, display the message "File not found".
5. Write a program that uses try, except, and else.
Take a number as input and print its square only if no error occurs.

6. Write a program that demonstrates the finally block.

Print a message saying "Program completed" whether an error occurs or not.

7. Write a program that raises a custom error if the user enters a negative age.

Display the message "Age cannot be negative".

8. Write a program that asks the user for two numbers and safely performs division.

Handle both invalid input and division by zero using multiple except blocks.

Start coding or [generate](#) with AI.